No: Date: / /

Comparison between various architecture types

(A) Stack Architecture

* Instructions like ADD, SUB, MUL do not require any memory addresses or register names.
* These instructions are called 0-Address instructions. because the CPU automatically assumes it needs to operate on whatever two numbers are on Top of Stack (TOS).

PUSH X : Finds the data from the RAM and place it on top of the stack.

POP Z : Take the top number off the stack and saves it into the RAM.

Ex:- Z = X + Y

```
PUSH X
PUSH Y
ADD   → X+Y
POP Z → Z = (X+Y)
```

Ex:- Y = A/B - (A-C*B)

* Left to right starts from

```
PUSH A
PUSH B
DIV:   → A/B
PUSH A
PUSH C
PUSH B
MUL    → C x B
SUB    → A - (C x B)
SUB    → (A/B) - (A - (C x B))
POP Y
```

The stack processor runs this sequence: PDF

Plaintext

```
PUSH A      // Stack: [A]
PUSH B      // Stack: [B, A]
DIV         // Stack: [ (A/B) ]

PUSH A      // Stack: [A, (A/B)]
PUSH C      // Stack: [C, A, (A/B)]
PUSH B      // Stack: [B, C, A, (A/B)]
MUL         // Multiplies C*B -> Stack: [ (C*B), A, (A/B) ]
SUB         // Subtracts (A - C*B) -> Stack: [ (A - C*B), (A/B) ]
SUB         // Subtracts (A/B) - (A - C*B) -> Stack: [ Final Result ]

POP Y       // Saves Final Result to RAM location Y
```

No: _____ Date: __/__/____

(B) Accumulator Architecture:

* Used in older computers
* Can hold one number when executing.

LOAD x : Copies data from the RAM to the Accumulator (ACC = Mem[x])

STORE Y : Copies the current value in the Accumulator to the RAM (Mem[Y] = ACC)

Ex:- Z = X + Y

LOAD X
ADD Y → X + Y
STORE Z → Z = (X+Y)

Ex:- Y = A/B - (A - C*B)

Starts from Right to left

LOAD C
MUL B → C * B
STORE D → D = (C * B)
LOAD A
SUB D → A - D
STORE D → D = (A - D)
LOAD A
DIV B → A/B
SUB D → (A/B) - D
STORE Y → Y = (A/B) - D

Y = (A/B) - (A - C*B)

A temp location in the RAM

Plaintext

```
// Part 1: Calculate (C * B)
LOAD C       // ACC = C
MUL B        // ACC = C * B
STORE D      // Save temporary result (C * B) in RAM location D

// Part 2: Calculate (A - D)  ->  which is (A - C * B)
LOAD A       // ACC = A
SUB D        // ACC = A - D
STORE D      // Save temporary result (A - C * B) in RAM location D

// Part 3: Calculate (A / B) - D
LOAD A       // ACC = A
DIV B        // ACC = A / B
SUB D        // ACC = (A / B) - D
STORE Y      // Save final answer to RAM location Y
```

No: _____ Date: __/__/____

(c) Memory - Memory architecture.

* This uses 2 - Address instruction formal as well as 3 - Address instruction format.

* 3 - Address instructions

Format : OPERAND Destination, SOURCE1, SOURCE2

Ex:- ADD Z, X, Y

Reads from X and Y

Stores in Z

* 2 - Address instructions

Format : OPERAND Destination and SOURE1, SOURCE2

Ex:- ADD X, Y

Reads from X and Y and overwrites X

Ex:- $Y = A/B - (A - C * B)$

3 - operand memory seguence

DIV D, A, B → D = A / B

MUL E, C, B → E = C * B

SUB E, A, E → E = A - E

SUB Y, D, E → Y = D - E

3-Operand Memory Sequence

Plaintext

```
DIV D, A, B    // Mem[D] = Mem[A] / Mem[B]
MUL E, C, B    // Mem[E] = Mem[C] * Mem[B]
SUB E, A, E    // Mem[E] = Mem[A] - Mem[E]
SUB Y, D, E    // Mem[Y] = Mem[D] - Mem[E]
```

No: _____ Date: __/__/____

(b) LOAD - STORE Architecture.

* This is the standard model used in all modern processors.
* The design follows one golden rule: Arithmetic & logical calculations are never allowed to touch RAM directly.
* General Purpose Registers are used.

Ex:- $Z = X + Y$

```
LOAD R1, X
LOAD R2, Y
ADD  R3, R1, R2  → R3 = R1 + R2
STORE Z, R3
```

Ex:- $Y = A/B - (A - C*B)$

```
LOAD R1, A
LOAD R2, B
LOAD R3, C

DIV  R4, R1, R2  → R4 = A/B
MUL  R5, R3, R2  → R5 = C*B
SUB  R5, R1, R5  → R5 = A - (C*B)
SUB  R6, R4, R5  → R6 = (A/B) - (A - (C*B))

STORE Y, R6
```

Plaintext

```
// Step 1: Load all required raw variables from RAM into registers
LOAD R1, A        // R1 = Mem[A]
LOAD R2, B        // R2 = Mem[B]
LOAD R3, C        // R3 = Mem[C][cite: 1]

// Step 2: Perform all math purely inside registers
DIV R4, R1, R2    // R4 = A / B[cite: 1]
MUL R5, R3, R2    // R5 = C * B[cite: 1]
SUB R5, R1, R5    // R5 = A - (C * B)[cite: 1]
SUB R4, R4, R5    // R4 = (A / B) - (A - C * B)[cite: 1]

// Step 3: Write the final calculated result back to RAM
STORE Y, R4       // Mem[Y] = R4[cite: 1]
```